********** List Data Structures **********

In [ ]:

List Data Structures

```
A list is a container which holds comma - seperated values between Square Brackets []
where items or elements need not all have the same type. it can have any number of ite
ms
and they may be of different type (integer, float, strings etc.)

List has the following five characteristics:
    1.mutable
    2.ordered data structure
    3.mixed type elements
    4.variable length
    5.zero based indexing.

  Types of Lists:
    1.empty list
    2.numeric list
    3.float list
    4.string list
    5.mixed list
    6.nested list
```

In [ ]:

Epty List

which list dose not have any values

In [2]:
```
empt=[]
print(empt)
```

```
[]
```

In [ ]:

Number List

Which List consist of only Numeric values

In [3]:
```
num=[1,2,3,4,5]
print(num)
```

```
[1, 2, 3, 4, 5]
```

In [ ]:

Float List

Which List consist of only Float values

In [4]:
```
flt=[2.6,4.5,5.6,7.7,10.8]
print(flt)
```

```
[2.6, 4.5, 5.6, 7.7, 10.8]
```

In [ ]:

String List

Which List consist of only String values

In [5]:
```
str=["John","mike","sam","smith"]
print(str)
```

```
['John', 'mike', 'sam', 'smith']
```

In [ ]:

Mixed List

Which List consist of all types of values (int,float,str)

In [6]:
```
mix=[1,2,3,2.5,2.6,4.8,"A","B","C"]
print(mix)
```

```
[1, 2, 3, 2.5, 2.6, 4.8, 'A', 'B', 'C']
```

In [ ]:

Nested

Which List contains a List inside a List

In [7]:
```
nst=[1,2,3,4,[2.5,3.6,4.8,5.9],["A","B","C","D"]]
print(nst)
```

```
[1, 2, 3, 4, [2.5, 3.6, 4.8, 5.9], ['A', 'B', 'C', 'D']]
```

In [ ]:

Basic List Operations: we can perform the following basic operations on list data structure.

len():

it will give the length of the list (no.of elements in the list)

In [1]:
```
pylist=[1,2,3,4,5]
print(len(pylist))
```

```
5
```

In [ ]:

concatenation:

it will join two lists

In [2]:
```
# concatenation: it will join two lists
pystr=[1,2,3,4,5]
pystr2=[6,7,8,9,10]
pystr3=pystr+pystr2
print(pystr3)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

In [ ]:

repetition:

it will repeate the string objects how many times you want.

In [3]:
```
pystr=[1,2,3,4,5]
print(pystr*5)
```

```
[1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
```

In [ ]:

membership:

it will check wheather the element in the list or not.

In [4]:
```
pystr=[1,2,3,4,5]
print(2 in pystr)
```

```
True
```

In [ ]:

min():

it will returns the minimum value in the list.

In [5]:
```
bigdata=["big data","hadoop","spark","tableau"]
print(min(bigdata))
```

```
big data
```

In [ ]:

max():

it will returns the maximum value in the list.

In [6]:
```
bigdata=["big data","hadoop","spark","tableau"]
print(max(bigdata))
```

```
tableau
```

In [ ]:

sum():

it is used to display sum of values in the list, only for numeric values.

In [7]:
```
pylist=[1,2,3,4,5]
print(sum(pylist))
```

```
15
```

In [ ]:

Python List/Array Methods

In [ ]:
```
Python List/Array Methods
Python has a set of built-in methods that you can use on lists/arrays.

Method  Description
append()        Adds an element at the end of the list
clear()         Removes all the elements from the list
copy()          Returns a copy of the list
count()         Returns the number of elements with the specified value
extend()        Add the elements of a list (or any iterable), to the end of the current list
index()         Returns the index of the first element with the specified value
insert()        Adds an element at the specified position
pop()   Removes the element at the specified position
remove()        Removes the first item with the specified value
reverse()       Reverses the order of the list
sort()  Sorts the list
```

In [ ]:

append()

Adds an element at the end of the list

In [8]:
```
fruits = ['apple', 'banana', 'cherry']
fruits.append("orange")
print(fruits)
```

```
['apple', 'banana', 'cherry', 'orange']
```

In [ ]:

clear()

Removes all the elements from the list

In [9]:
```
fruits = ['apple', 'banana', 'cherry', 'orange']
fruits.clear()
print(fruits)
```

```
[]
```

In [ ]:

copy()

Returns a copy of the list

In [10]:
```
fruits = ['apple', 'banana', 'cherry', 'orange']
x = fruits.copy()
print(x)
```

```
['apple', 'banana', 'cherry', 'orange']
```

In [ ]:

count()

Returns the number of elements with the specified value

In [11]:
```
fruits = ['apple', 'cherry', 'banana', 'cherry']
print(fruits.count("cherry"))
```

```
2
```

In [ ]:

extend()

Add the elements of a list (or any iterable), to the end of the current list

In [12]:
```
pystr2=[1,2,3,4,5]
pystr3.extend([6,7,8])
print(pystr1)
```

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

In [ ]:

index()

Returns the index of the first element with the specified value

In [15]:
```
fruits = ['apple', 'banana', 'cherry']
print(fruits.index("cherry"))
```

```
2
```

In [ ]:

insert()

Adds an element at the specified position

In [16]:
```
fruits = ['apple', 'banana', 'cherry']
fruits.insert(1, "orange")
print(fruits)
```

```
['apple', 'orange', 'banana', 'cherry']
```

In [ ]:

pop()

Removes the element at the specified position

In [17]:
```
fruits = ['apple', 'banana', 'cherry']
print(fruits.pop(1))
```

```
banana
```

In [ ]:

remove()

Removes the first item with the specified value

In [18]:
```
fruits = ['apple', 'banana', 'cherry']
fruits.remove("banana")
print(fruits)
```

```
['apple', 'cherry']
```

In [ ]:

reverse()

Reverses the order of the list

In [19]:
```
fruits = ['apple', 'banana', 'cherry']
fruits.reverse()
print(fruits)
```

```
['cherry', 'banana', 'apple']
```

In [ ]:

sort()

it will Sorts the list

In [22]:
```
cars = ['Ford', 'BMW', 'Volvo']
cars.sort()
print(cars)
```

```
['BMW', 'Ford', 'Volvo']
```

In [ ]:

Built-in Functions with list:

all():

it returns true when all elements in the given iterable are true. if not, it returns false.

In [ ]:
```
# all(): it returns true when all elements in the given iterable are true. if not, it return
s false.
            # all values are true = True
            # all values are false = False
            # one value is true(others are false) = False
            # one value is false(others are true) = False

pyall=[1,2,3,4]
print(all(pyall)) # True
pyall=[0,"False"]
print(all(pyall)) # False
pyall=[0,"False",5]
print(all(pyall)) # False
```

In [ ]:

any():

it returns true if any element of an iterable as true. if not, it returns false.

In [ ]:
```
# any(): it returns true if any element of an iterable as true. if not, it returns false.
            # all values are true = True
            # all values are false = False
            # one value is true(others are false) = True
            # one value is false(others are true) = True

pyany=[1,2,3,4]
print(any(pyany)) # True
pyall=[0,"False"]
print(any(pyany)) # False
pyall=[0,"False",5]
print(any(pyany)) # True
```

In [ ]:

enumerate():(item auto index)

it adds counter to an iterable and returns it(the enumerate object)

In [20]:
```
# syntax: enumerate(iterable,start=0)

#Example-1

bigdata=["big data","hadoop","spark","tableau"]
edata=enumerate(bigdata)
print(type(edata))
print(list(edata))

<class 'enumerate'>
[(0, 'big data'), (1, 'hadoop'), (2, 'spark'), (3, 'tableau')]
```

In [21]:
```
#Example-2

bigdata=["big data","hadoop","spark","tableau"]
edata=enumerate(bigdata,500)
print(list(edata))

[(500, 'big data'), (501, 'hadoop'), (502, 'spark'), (503, 'tableau')]
```

In [ ]:
```
# output of the given script

pylist=[]

for i in range(10):
    if i%2==1:
        pylist.append(i)

print(pylist)
```

In [ ]:

List Comprehension:

In this method, we basically define the process through which the list needs to be created. we specify what operation needs to be done and on what element needs to done. syntax: [expression "for" item in iterable]

In [22]:
```
# Example-1

print([x*2 for x in[1,2,3,4,5,6]])
```

```
[2, 4, 6, 8, 10, 12]
```

In [23]:
```
# Example-2

print([x for x in[2,3,4,5,6] if x%2==0])
```

```
[2, 4, 6]
```

Syntax: [expression if conditional else statement for item in iterable]

In [24]:
```
# Example-3

print([x if x<3 else x+1 for x in[1,2,3,4,5,6]]) # [1, 2, 4, 4, 5, 6]
```

```
[1, 2, 4, 4, 5, 6]
```

In [25]:
```
# Example-4

print([x if x<3 else x+1 for x in[1,2,3,4,5,6]]) # [4, 5, 3, 4, 5, 6]
```

```
[4, 5, 3, 4, 5, 6]
```

In [26]:
```
# Example :

sample=["Data Science","Machine Learning","Hadoop","Spark"]
print([item [0] for item in sample])
```

```
['D', 'M', 'H', 'S']
```

In [ ]: